

BMI 500 Scanpy Profiling Report

PBMC3K, PBMC6K, and PBMC10K

Objective. Profile the Scanpy PBMC workflow at three dataset sizes using coarse-grain, instrumented, and fine-grain methods; identify performance bottlenecks; visualize scaling; and estimate expected runtimes at 20K cells.

1. Methods

- Coarse-grain profiling:** `/usr/bin/time -v` was used to measure whole-program wall time, CPU usage, user/system CPU time, and peak resident memory.
- Instrumented profiling:** `time.time()` was added around each meaningful ``#%%`` code block to measure section-level runtime.
- Fine-grain profiling:** `cProfile` was applied to `sc.tl.rank_genes_groups(...)` to identify internal runtime costs for the Wilcoxon ranking path.
- Cluster execution:** Each dataset was run as a Slurm batch job with 4 requested CPU cores and 8 GB of memory. All three jobs completed with exit status 0.

2. Coarse-Grain Profiling

Dataset	User time	System time	CPU	Wall time	Peak memory
PBMC3K	25.66 s	1.52 s	103%	26.33 s	892,652 KB
PBMC6K	35.35 s	2.00 s	103%	35.98 s	924,912 KB
PBMC10K	75.61 s	5.78 s	109%	74.30 s	1,359,452 KB

Interpretation. Wall time increased from 26.33 s at 3K cells to 74.30 s at 10K cells, while peak memory rose from about 0.89 GB to about 1.36 GB. CPU utilization stayed near 100% even though four CPU cores were requested, so the complete workflow did not sustain four-core parallel execution.

3. Console Output Highlights

PBMC3K

```
User time (seconds): 25.66
System time (seconds): 1.52
Percent of CPU this job got: 103%
Elapsed (wall clock) time: 26.33 s
Maximum resident set size (kbytes): 892652
Exit status: 0
```

PBMC6K

```
User time (seconds): 35.35
System time (seconds): 2.00
Percent of CPU this job got: 103%
Elapsed (wall clock) time: 35.98 s
Maximum resident set size (kbytes): 924912
Exit status: 0
```

PBMC10K

```
User time (seconds): 75.61
System time (seconds): 5.78
Percent of CPU this job got: 109%
Elapsed (wall clock) time: 74.30 s
```

Maximum resident set size (kbytes): 1359452
Exit status: 0

4. Instrumented Profiling

Section	PBMC3K	PBMC6K	PBMC10K
Normalization	0.0445 s	0.0700 s	0.4468 s
Highly variable genes	0.0692 s	0.0788 s	0.4595 s
Scaling	0.0413 s	0.0743 s	0.1706 s
PCA	0.3378 s	0.5599 s	2.0478 s
Neighbors	4.1083 s	4.0346 s	30.5457 s
Louvain clustering	0.1267 s	0.3582 s	0.8066 s
UMAP	8.4489 s	15.8173 s	14.1890 s
Write output	0.1661 s	0.2568 s	0.6890 s
Rank genes	1.9042 s	3.5704 s	13.5918 s

Major bottlenecks. UMAP was the largest cost for PBMC3K and PBMC6K, while neighborhood graph construction became the dominant cost at PBMC10K (30.55 s). Gene ranking also grew sharply, reaching 13.59 s at PBMC10K. Basic preprocessing remained comparatively inexpensive.

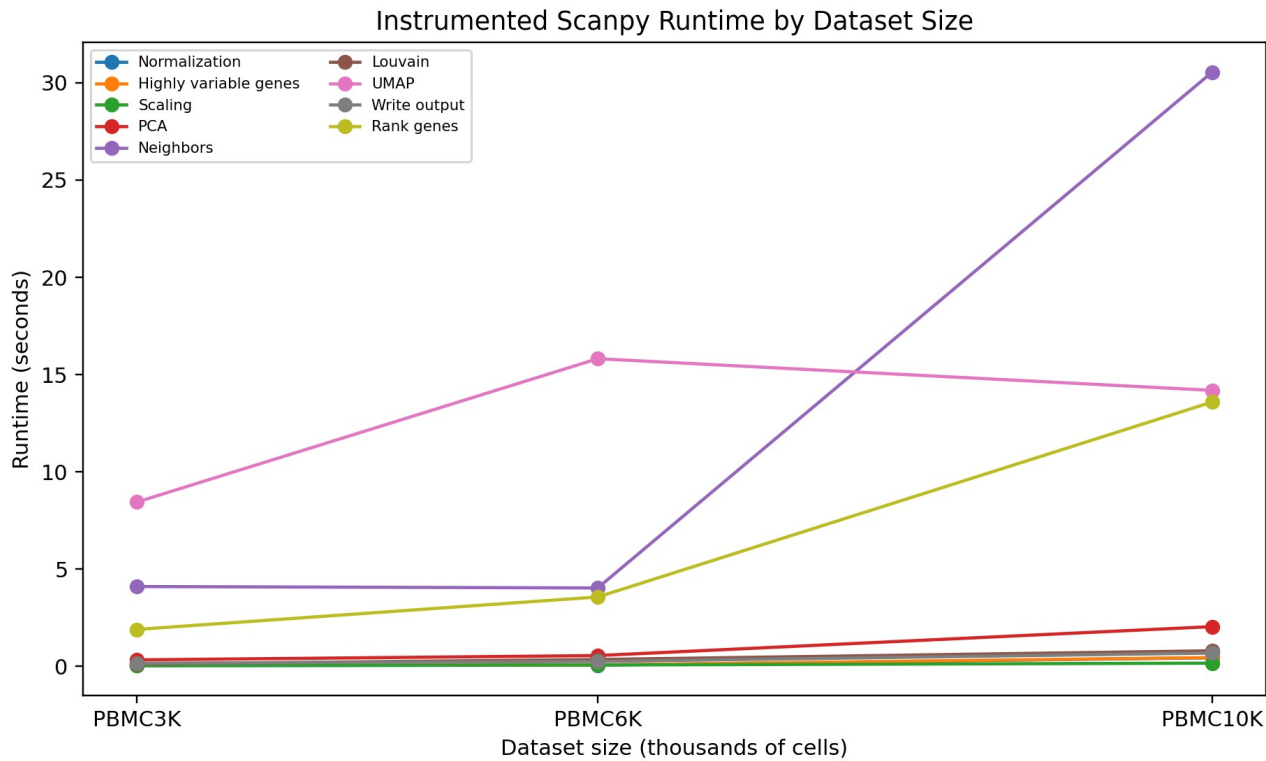


Figure 1. Instrumented runtime by dataset size. Lines connect measured values and are not assumed to represent a smooth theoretical scaling law.

5. Fine-Grain cProfile Results

Metric / function	PBMC3K	PBMC6K	PBMC10K
Function calls	42,515	80,856	308,369
rank_genes_groups	1.727 s	3.397 s	13.573 s
compute_statistics	1.673 s	3.307 s	13.161 s
wilcoxon	1.624 s	3.251 s	13.040 s
_ranks	1.332 s	2.609 s	8.653 s

_basic_stats	0.126 s	0.245 s	2.767 s
--------------	---------	---------	---------

Interpretation. The total profiled runtime for rank_genes_groups increased from 1.727 s at 3K to 13.573 s at 10K. Most cumulative time flowed through compute_statistics and the Wilcoxon path. The internal _ranks operation was a major contributor, and the total function-call count increased more than sevenfold from 3K to 10K.

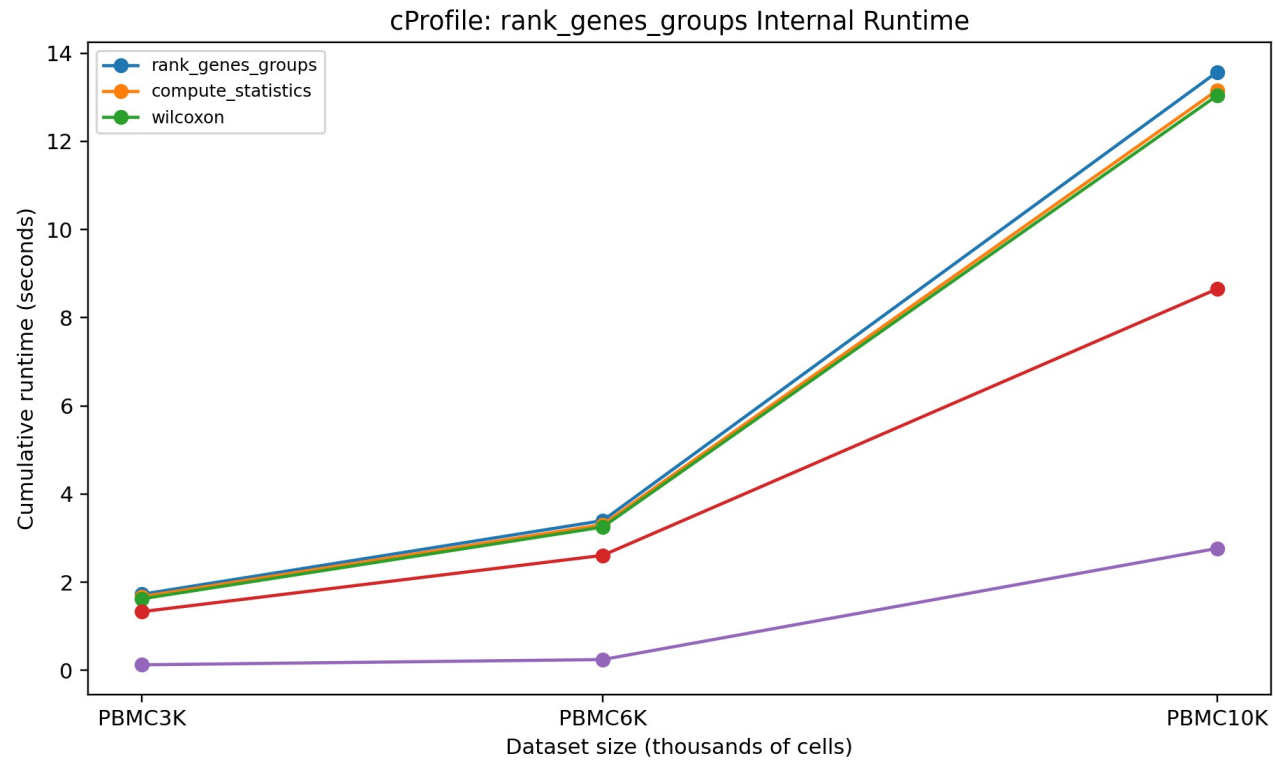


Figure 2. cProfile cumulative runtimes for major functions inside rank_genes_groups across the three datasets.

6. Estimated PBMC20K Runtime

Section	Estimated 20K runtime
Normalization	~1.00 s
Highly variable genes	~0.99 s
Scaling	~0.35 s
PCA	~4.41 s
Neighbors	~66.62 s
Louvain clustering	~1.77 s
UMAP	~23.12 s
Write output	~1.42 s
Rank genes	~29.79 s

Method and limitation. These estimates use a simple linear fit through the measured 3K, 6K, and 10K timings. They are approximate projections, not measured 20K results. Some stages, especially Neighbors, clearly show nonlinear behavior over the observed range.

7. Conclusions

The workflow does not scale uniformly. At smaller sizes, UMAP dominates runtime; by PBMC10K, neighbor graph construction becomes the largest bottleneck. Gene ranking also grows rapidly and its cProfile results show that Wilcoxon rank computation is the main internal cost. CPU utilization remains near one fully utilized core on average, so simply requesting more CPU cores did not materially accelerate the entire pipeline.

Appendix A. Slurm Batch Scripts

pbmc3k.sh

```
#!/bin/bash
#SBATCH --job-name=pbmc3k
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --output=pbmc3k_output.txt

cd ~/bmi500
source bmi500/bin/activate

/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc3k \
  --out-dir data \
  --num-threads 4
```

pbmc6k.sh

```
#!/bin/bash
#SBATCH --job-name=pbmc6k
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --output=pbmc6k_output.txt

cd ~/bmi500
source bmi500/bin/activate

/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc6k \
  --out-dir data \
  --num-threads 4
```

pbmc10k.sh

```
#!/bin/bash
#SBATCH --job-name=pbmc10k
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --output=pbmc10k_output.txt

cd ~/bmi500
source bmi500/bin/activate

/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc10k \
  --out-dir data \
  --num-threads 4
```